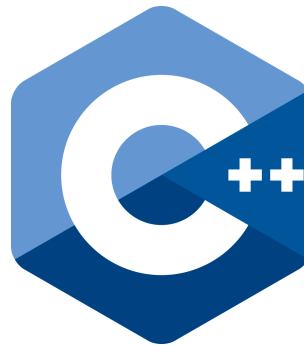




Seminario de Programación



De C a C++ y Fundamentos de Programación Orientada a Objetos



Patricio Olivares

Ingeniería Civil Telemática
Departamento de Electrónica
Universidad Técnica Federico Santa María

Introducción a C++

1. Breve historia

- **C (1972, Dennis Ritchie, Bell Labs):** Lenguaje de propósito general, cercano al hardware, con control detallado de memoria. Base de sistemas operativos (Unix), compiladores, y gran parte de la infraestructura digital.
- **C++ (1983, Bjarne Stroustrup, también en Bell Labs):** Nace como "C con clases", con el objetivo de **agregar abstracción sin perder eficiencia**. Evolucionó hasta convertirse en un lenguaje multiparadigma (procedural, orientado a objetos, genérico y funcional). Hoy en día es estándar ISO (últimas versiones: C++11, C++14, C++17, C++20, C++23).

C++ **no reemplaza a C**, sino que lo extiende con nuevas formas de programar.

Instalación de C++ en Linux

1. Verificar si ya está instalado

La mayoría de las distribuciones traen **gcc/g++** preinstalados. En una terminal:

```
gcc --version  
g++ --version
```

Si retorna la versión correspondiente, significa que ya tienes los compiladores instalados.

2. Instalar en distribuciones basadas en Debian/Ubuntu

```
sudo apt update  
sudo apt install build-essential
```

Esto instala:

- `gcc` y `g++`
- `make`
- Librerías estándar de C/C++

2. Diferencias principales entre C y C++

Compilación

- **C:** usa `gcc`, estándar ANSI C o C99.
- **C++:** usa `g++`, con soporte de múltiples estándares.
- Ambos generan código máquina eficiente, pero C++ agrega un compilador más complejo debido a plantillas, sobrecarga de operadores, etc.

Diferencia en la compilación de C vs C++

- **Lenguaje C:** Se compila con `gcc`, por ejemplo:

```
gcc programa.c -o programa  
./programa
```

El compilador espera código en C, que sigue reglas más simples (sin clases, sin `iostream`, etc.).

- **Lenguaje C++:** Se compila con `g++`, que entiende extensiones y características de C++ (clases, plantillas, `iostream`, etc.).

```
g++ programa.cpp -o programa  
./programa
```

Aunque `g++` también puede compilar muchos programas en C, **la librería estándar y las características del lenguaje cambian**: en C++ se soporta POO, plantillas, `std::vector`, etc.

Ejemplo de Compilación: "Hola Mundo" en C vs C++

En C

Código (`hola.c`):

```
#include <stdio.h>

int main() {
    printf("Hola Mundo desde C\n");
    return 0;
}
```

Compilación y ejecución:

```
gcc hola.c -o hola_c
./hola_c
```

En C++

Código (`hola.cpp`):

```
#include <iostream>

int main() {
    std::cout << "Hola Mundo desde C++" << std::endl;
    return 0;
}
```

Compilación y ejecución:

```
g++ hola.cpp -o hola_cpp
./hola_cpp
```

Diferencias

- **Librerías:**
 - C usa `<stdio.h>` con funciones (`printf` , `scanf`).
 - C++ usa `<iostream>` con objetos (`std::cout` , `std::cin`).
- **Estilo:**
 - C está basado en llamadas a funciones.
 - C++ es orientado a objetos y permite sobrecarga de operadores.
- **Compilador:**

- C: `gcc`.
- C++: `g++` (aunque `g++` también puede compilar código C).

Más Diferencias...

Entrada y salida

- C: `printf`, `scanf` (orientados a formato, basados en funciones de la librería estándar).
- C++: `cin`, `cout` de la librería `<iostream>`, orientados a objetos y más seguros.

```
// C
int x;
scanf("%d", &x);
printf("Valor: %d\n", x);
// C++
int x;
std::cin >> x;
std::cout << "Valor: " << x << std::endl;
```

C++ evita errores de formato, integra mejor con tipos definidos por el usuario.

Manejo de memoria

- C: uso de `malloc` y `free`.
- C++: operadores `new` y `delete`.

```
// C
// Solicitud de espacio en memoria
int* p = (int*)malloc(sizeof(int));
*p = 10; // uso
// Una vez utilizado, se libera dicho espacio
free(p);
p = NULL; // Asignación a NULL para buena práctica
// C++
// Solicitud de espacio en memoria (new)
int *p = new int;
*p = 10; // uso (similar a C)
// Liberación del sector de memoria una vez utilizado
delete p;
p = NULL;
```

También, a partir de C++11 se utilizan *smart pointers* (no vistos en esta asignatura).

Arreglos y colecciones

- C: arrays estáticos o dinámicos (con `malloc`).

- **C++:** múltiples tipos de contenedores ya disponibles desde la STL (Standard Template Library), incluyendo `std::vector`.

```
// C
int* arr = (int*)malloc(3 * sizeof(int));
arr[0] = 10; arr[1] = 20; arr[2] = 30;
free(arr);
// C++
// Ojo, requerida la inclusión para su uso
#include <vector>

std::vector<int> arr = {10, 20, 30};
// Gestión automática de memoria
// Para recorrer el vector, también se hace a través de la notación
// de índice
// arr[0] = 10; arr[1] = 20; etc...
```

C++ ofrece mayor seguridad, flexibilidad, y más funciones ya implementadas.

Struct vs Class

- **C:** `struct` solo agrupa datos, sin funciones ni encapsulación.
- **C++:** `struct` y `class` permiten atributos y métodos, constructores y destructores, encapsulación y herencia (conceptos de Programación Orientada a Objetos).

```
// C
struct Persona {
    char nombre[50];
    int edad;
};
// C++
class Persona {
private:
    std::string nombre;
    int edad;
public:
    Persona(std::string n, int e) : nombre(n), edad(e) {}
    void saludar() { std::cout << "Hola, soy " << nombre; }
};
```

3.Programación Orientada a Objetos (POO)

- Los programas crecen en complejidad y se vuelven difíciles de mantener con código estructurado solo con funciones y datos separados.
- Necesitamos un **modelo que se acerque más al mundo real**: entidades con *características y comportamientos*.

- La POO propone organizar el software en **objetos** que colaboran entre sí.

1. Conceptos básicos

- **Objeto:** entidad que representa algo del mundo real o abstracto.
 - **Atributos (estado):** información que describe al objeto.
 - **Métodos (comportamiento):** acciones que puede realizar.
- **Clase:** molde o plantilla para crear objetos. Define los atributos y métodos comunes.

Ejemplo conceptual:

- Objeto: *Auto rojo, patente ABCD12.*
- Clase: *Auto.* Tiene atributos (color, patente) y métodos (acelerar, frenar).

2. Ventajas de la POO

- **Reutilización de código:** clases pueden usarse en distintos proyectos.
- **Mantenibilidad:** más fácil de entender y modificar.
- **Extensibilidad:** se pueden agregar nuevas clases sin reescribir todo.
- **Modelado natural:** refleja mejor los problemas reales.

3. Ejemplo

Imaginemos un videojuego con personajes:

- Cada personaje tiene atributos: nombre, vida, posición.
- Cada personaje tiene métodos: moverse, atacar, defenderse.
- Podemos definir una clase base *Personaje* y luego derivar *Guerrero*, *Mago*, *Arquero*.

Esto permite representar el mundo del juego de manera ordenada, coherente y extensible.

4. Programación Orientada a Objetos en C++

1. Clases y Objetos en C++

- En C++, una **clase** es la construcción que permite encapsular atributos y métodos.
- Un **objeto** es una instancia de esa clase.
- Sintaxis similar a `struct`, pero con más funcionalidades.

Ejemplo simple:

```
#include <iostream>
#include <string>

class Persona {
public:
    std::string nombre;
    int edad;

    void saludar() {
        std::cout << "Hola, soy " << nombre << std::endl;
    }
};

int main() {
    Persona p1;
    p1.nombre = "Valeria";
    p1.edad = 20;
    p1.saludar();
}
```

2. Constructores y Destruidores

- **Constructor:** método especial que inicializa un objeto al ser creado.
- **Destructor:** método especial que se ejecuta antes de que el objeto sea destruido (libera memoria, recursos).

```
#include <iostream>
#include <string>

class Animal {
public:
    std::string nombre;

    Animal(std::string n){
        nombre = n;
        std::cout << "Creando animal " << nombre << std::endl;
    }

    ~Animal() {
        std::cout << "Destruyendo animal " << nombre << std::endl;
    }
};

int main() {
    Animal a("Tigre");
}
```

3. Listas de inicialización

En C++, los **constructores** pueden usar una **lista de inicialización** para asignar valores a los miembros **antes de ejecutar el cuerpo** del constructor.

- Inician directamente los atributos (más eficiente que asignar dentro del cuerpo).
- Son **obligatorias** para miembros `const`, referencias o clases sin constructor por defecto.
- Permiten inicializar clases base (concepto de herencia).

Sintaxis

```
Clase::Clase(tipo1 a, tipo2 b)
    : atributo1(a), atributo2(b) // lista de inicialización
{
    // cuerpo del constructor
}
```

Ejemplo básico

```
#include <iostream>
#include <string>

class Persona {
    const int id;
    std::string nombre;
public:
    Persona(int i, std::string n)
        : id(i), nombre(n) { // inicialización directa
        std::cout << "Persona creada\n";
    }

    void mostrar() {
        std::cout << id << ": " << nombre << std::endl;
    }
};

int main() {
    Persona p(1, "Valeria");
    p.mostrar();
}
```

- Los miembros se inicializan **en el orden en que se declaran**, no en el de la lista.
- Prefiere inicializar en la lista antes que asignar en el cuerpo.

4. Modificadores de acceso y encapsulación

Definen qué partes del programa pueden usar los atributos o métodos.

- **public:** accesible desde cualquier parte.

- **private:** accesible solo dentro de la clase.
- **protected:** accesible en la clase y en clases derivadas (asociado a herencia, concepto posterior).

```
class CuentaBancaria {
private:
    double saldo;

public:
    CuentaBancaria(double s) : saldo(s) {}
    void depositar(double monto) { saldo += monto; }
    double obtenerSaldo() { return saldo; }
};
```

Utilizar modificadores de acceso, permite **encapsular** elementos, para proteger datos internos evitando que se modifique un estado de manera incorrecta.

Ejemplo: `CuentaBancaria` no expone directamente su saldo, sino que obliga a usar métodos (`depositar`, `obtenerSaldo`).

5. Herencia, Polimorfismo y Métodos Virtuales en C++

5.1. Herencia

- Permite que una clase "derivada" extienda a otra clase "base".
- La clase derivada hereda atributos y métodos, y puede añadir nuevos o redefinir los existentes.
- Ventaja: reutilización y organización del código.

Ejemplo en C++

```
#include <iostream>
#include <string>

// Clase base
class Animal {
public:
    std::string nombre;

    Animal(std::string n) : nombre(n) {}
    void mover() { std::cout << nombre << " se mueve." <<
std::endl; }
};

// Clase derivada
class Perro : public Animal {
public:
    Perro(std::string n) : Animal(n) {}
```

```

    void ladrar() { std::cout << nombre << " dice: Guau!" <<
std::endl; }
};

int main() {
    Perro p("Firulais");
    p.mover();    // Heredado de Animal
    p.ladrar();   // Método propio de Perro
}

```

5.2. Polimorfismo

Concepto

- "Muchas formas": diferentes clases responden de manera distinta al mismo mensaje.
- Ejemplo: todos los animales tienen el método `comer()`, pero cada uno lo hace distinto.

Ejemplo en C++ (sin virtual):

```

class Animal {
public:
    void comer() { std::cout << "El animal come." << std::endl; }
};

class Lobo : public Animal {
public:
    void comer() { std::cout << "El lobo come carne." << std::endl; }
};

int main() {
    Animal* a = new Lobo();
    a->comer(); // Imprime "El animal come." (¡no lo esperado!)
    delete a;
}

```

Problema: al llamar desde un puntero a la clase base, **se ignora la redefinición** en la subclase.

5.3. Métodos Virtuales

Concepto

- Permiten que una función redefinida en una clase hija sea llamada correctamente, incluso a través de un puntero o referencia de la clase base.
- Implementan el polimorfismo dinámico.

Ejemplo en C++ con `virtual`:

```

class Animal {
public:
    virtual void comer() { // Método virtual
        std::cout << "El animal come." << std::endl;
    }
};

class Lobo : public Animal {
public:
    void comer() { // Sobrescribe el método
        std::cout << "El lobo come carne." << std::endl;
    }
};

class Pez : public Animal {
public:
    void comer() {
        std::cout << "El pez come plancton." << std::endl;
    }
};

int main() {
    Animal* animales[3];
    animales[0] = new Animal();
    animales[1] = new Lobo();
    animales[2] = new Pez();

    for (int i = 0; i < 3; i++) {
        animales[i]->comer(); // Llama al método correcto según el
objeto real
        delete animales[i];
    }
}

```

Salida:

```

El animal come.
El lobo come carne.
El pez come plancton.

```

5.4. Métodos Virtuales Puros

Concepto

- Son métodos que no se implementan en la clase base y que deben ser implementados obligatoriamente en las clases derivadas, definiendo así una interfaz común.
- Una clase que contiene al menos un método virtual puro se denomina **clase abstracta** y **no puede instanciarse**.
- Se declaran utilizando `= 0` al final de la definición del método.

Ejemplo en C++ con método virtual puro (= 0):

```

#include <iostream>
#include <cmath>

class Figura {
public:
    virtual double area() = 0;    // Método virtual puro (sin
    // implementación)
    virtual void mostrar() = 0;  // Otro método virtual puro
};

class Circulo : public Figura {
private:
    double radio;
public:
    Circulo(double r) : radio(r) {}
    double area() { return M_PI * radio * radio; }
    void mostrar() {
        std::cout << "Círculo de radio " << radio << std::endl;
    }
};

class Rectangulo : public Figura {
private:
    double ancho, alto;
public:
    Rectangulo(double a, double b) : ancho(a), alto(b) {}
    double area() { return ancho * alto; }
    void mostrar() {
        std::cout << "Rectángulo de " << ancho << "x" << alto <<
std::endl;
    }
};

int main() {
    Figura* figuras[2];
    figuras[0] = new Circulo(3.0);
    figuras[1] = new Rectangulo(2.0, 5.0);

    for (int i = 0; i < 2; i++) {
        figuras[i]->mostrar();
        std::cout << "Área: " << figuras[i]->area() << std::endl;
        delete figuras[i];
    }

    // Figura f; // ERROR: no se puede instanciar una clase
    // abstracta
}

```

Salida:

Círculo de radio 3
 Área: 28.2743
 Rectángulo de 2x5
 Área: 10

Resumen

- Los métodos virtuales puros sirven para **definir comportamientos que deben implementarse en las clases derivadas**.
- Las clases con métodos virtuales puros **actúan como interfaces**.

6. Punteros y Memoria Dinámica en POO

En POO, los punteros se utilizan para **crear, enlazar y administrar objetos de forma dinámica**. Esto permite construir estructuras flexibles y modelar relaciones entre clases.

6.1 Creación dinámica de objetos

En lugar de crear objetos estáticos, podemos instanciarlos en tiempo de ejecución.

```
#include <iostream>
#include <string>

class Persona {
    std::string nombre;
public:
    Persona(std::string n) : nombre(n) {}
    void saludar() { std::cout << "Hola, soy " << nombre <<
std::endl; }
};

int main() {
    Persona* p = new Persona("Valeria");
    p->saludar();
    delete p; // liberar memoria
}
```

6.2 Arreglos dinámicos de objetos

También es posible reservar espacio para varios objetos de una clase.

```
class Punto {
public:
    int x, y;
    Punto(int a, int b) : x(a), y(b) {}
};

int main() {
```

```
Punto* arr = new Punto[3]{{1,2}, {3,4}, {5,6}};
std::cout << arr[1].x << ", " << arr[1].y << std::endl;
delete[] arr; // liberar arreglo
}
```

6.3 Relaciones entre objetos

Los punteros permiten representar asociaciones entre objetos, útiles en jerarquías y composición.

```
class Estudiante {
    std::string nombre;
public:
    Estudiante(std::string n) : nombre(n) {}
    void mostrar() { std::cout << "Estudiante: " << nombre <<
std::endl; }
};

class Curso {
    Estudiante* alumno;
public:
    Curso(std::string n) { alumno = new Estudiante(n); }
    ~Curso() { delete alumno; }
    void mostrarCurso() { alumno->mostrar(); }
};

int main() {
    Curso c("Ana");
    c.mostrarCurso();
}
```

Buenas prácticas

- Cada `new` debe tener su `delete`.
- Si se usa `new[]`, debe usarse `delete[]`.
- No acceder a punteros después de liberar la memoria.

7. Biblioteca Estándar (STL) y uso de namespace `std`

La **STL (Standard Template Library)** es una de las partes más potentes de C++.

Proporciona **estructuras de datos genéricas** (contenedores), **algoritmos reutilizables** y **herramientas de iteración** listas para usar, reduciendo el tiempo de desarrollo y los errores.

Los principales componentes son:

- **Contenedores:** estructuras como `vector`, `list`, `map`, `set`, `queue`, etc.
- **Iteradores:** objetos que permiten recorrer los elementos de un contenedor.
- **Algoritmos:** funciones genéricas como `sort()`, `find()`, `count()`, `reverse()`.
- **Funciones y objetos genéricos:** permiten operar de manera flexible con distintos tipos.

7.1 Uso de espacios de nombres (`namespace`)

Un **namespace** agrupa identificadores (funciones, clases, variables, etc.) bajo un mismo nombre para **evitar conflictos**.

La biblioteca estándar de C++ está contenida en el **namespace `std`**, abreviatura de *standard*. Por ejemplo, `std::cout`, `std::string` y `std::vector` pertenecen todos a `std`.

```
#include <iostream>
#include <string>

int main() {
    std::string nombre = "Valeria";
    std::cout << "Hola, " << nombre << std::endl;
}
```

```
using namespace std;
```

Aunque es común ver:

```
using namespace std;
```

esto **no se recomienda** en proyectos grandes o archivos con muchas librerías, porque puede provocar **conflictos de nombres**.

Buena práctica: utilizar siempre el prefijo `std::` para dejar claro de dónde proviene cada elemento.

7.2 Contenedores: `vector` y `map`

Los contenedores almacenan colecciones de elementos. Aquí veremos dos de los más utilizados: `vector` y `map`.

`vector`

Un `std::vector` es un **arreglo dinámico** que puede cambiar de tamaño automáticamente.

```
#include <iostream>
#include <vector>
```

```
#include <algorithm>

int main() {
    std::vector<int> datos = {5, 1, 3, 4, 2};

    std::sort(datos.begin(), datos.end()); // ordena de menor a mayor

    std::cout << "Datos ordenados: ";
    for (auto n : datos) {
        std::cout << n << " ";
    }
    std::cout << std::endl;
}
```

map

Un `std::map` almacena **pares clave-valor** ordenados automáticamente por la clave. Es muy útil cuando se necesita **buscar elementos rápidamente** o asociar información.

```
#include <iostream>
#include <map>
#include <string>

int main() {
    std::map<std::string, int> edades;

    edades["Ana"] = 20;
    edades["Luis"] = 22;
    edades["Valeria"] = 19;

    std::cout << "Edad de Luis: " << edades["Luis"] << std::endl;

    std::cout << "Listado de edades:\n";
    for (auto par : edades) {
        std::cout << par.first << " -> " << par.second <<
std::endl;
    }
}
```

`first` representa la **clave** y `second` el **valor**. En este ejemplo, las claves se ordenan alfabéticamente de manera automática.

7.3 Otros contenedores importantes

Además de `vector` y `map`, la STL ofrece muchas estructuras útiles:

- `std::list`: lista doblemente enlazada.
- `std::set`: conjunto de elementos únicos.
- `std::queue` y `std::stack`: estructuras tipo cola y pila.

- `std::pair` : par de valores (muy usado con `map`).

Estos contenedores pueden combinarse entre sí y aprovechar los algoritmos genéricos de la STL para tareas comunes como búsqueda, ordenamiento y conteo.

8. Ejercicios

8.1 Ejercicio 1: Traducir C a C++

Instrucciones: A continuación tienes un programa escrito en C que:

1. Reserva memoria dinámicamente para un arreglo de enteros.
2. Lee `n` valores desde teclado.
3. Calcula la suma total usando una función con punteros.
4. Muestra el promedio de los valores leídos.

Tu tarea es **reescribir este programa en C++ (sin POO)**, utilizando las herramientas que ya conoces:

- Usa `std::cin` y `std::cout` en lugar de `scanf` y `printf` .
- Reemplaza el uso de `malloc` y `free` , por `new` , `delete` y/o un contenedor de la STL (por ejemplo, `std::vector<int>`).
- Mantén una función que reciba punteros para calcular la suma.

```
/* Código en C */
#include <stdio.h>
#include <stdlib.h>

int sumar_enteros(int* p, int n) {
    int suma = 0;
    printf("Sumando los elementos del arreglo...\n");
    for (int i = 0; i < n; i++) {
        printf(" Elemento %d: %d\n", i + 1, p[i]);
        suma += p[i];
    }
    printf("Suma total: %d\n", suma);
    return suma;
}

int main() {
    int n;
    printf("Ingrese la cantidad de números: ");
    scanf("%d", &n);

    int* arr = (int*)malloc(n * sizeof(int));
    if (arr == NULL) {
        printf("Error: no se pudo reservar memoria.\n");
        return 1;
    }
}
```

```

    }

    printf("Ingrese %d números enteros:\n", n);
    for (int i = 0; i < n; i++) {
        printf("Número %d: ", i + 1);
        scanf("%d", &arr[i]);
    }

    int s = sumar_enteros(arr, n);

    double promedio = 0.0;
    if (n > 0) {
        promedio = (double)s / n;
    }

    printf("Promedio: %.2f\n", promedio);

    free(arr);
    printf("Memoria liberada. Programa finalizado.\n");

    return 0;
}

```

Mantén la función `sumar_enteros` pero adáptala para C++.

8.2 Ejercicio 2: Sistema de Biblioteca

Instrucciones: Una biblioteca necesita un sistema para gestionar sus libros, usuarios y préstamos. Debes **diseñar el modelo orientado a objetos** que representaría este sistema. Describe en texto lo siguiente:

1. Identifica las **clases principales** del dominio (mínimo tres).
2. Define sus **atributos** más importantes.
3. Especifica los **métodos** relevantes para cada clase.
4. Indica si existe alguna **relación de herencia** o **composición** entre ellas.
5. Explica brevemente cómo se utilizarían los objetos en un caso de uso típico (por ejemplo, cuando un usuario solicita un préstamo).

8.3 Ejercicio 3: Clase Persona

Crea una clase `Persona` con los atributos `nombre`, `edad` y `ciudad`. Incluye un método `presentarse()` que muestre un mensaje con esa información. En `main`, crea dos objetos `Persona` y llama al método de ambos.

8.4 Ejercicio 4: Clase Estudiante

Diseña una clase `Estudiante` que tenga:

- Un constructor que inicialice el nombre y la carrera.
- Un destructor que muestre un mensaje cuando el objeto se elimina.

Crea algunos objetos dentro de un bloque `{ }` para observar cuándo se llama el destructor.

8.5 Ejercicio 5: Animales

Usa herencia y polimorfismo: Crea una clase base `Animal` con un método virtual `sonido()`. Define dos clases derivadas `Perro` y `Gato`, que redefinan el método para mostrar sonidos distintos. Crea un arreglo de punteros a `Animal` y llama `sonido()` en cada uno.

8.6 Ejercicio 6: Figuras Geométricas

Utilizando el ejemplo visto en clase:

1. Agrega una nueva clase `Triangulo` que calcule el área con base y altura.
2. Crea un arreglo de punteros `Figura*` que contenga un `Circulo`, un `Rectangulo` y un `Triangulo`.
3. Muestra el área de cada figura.

Desafío adicional: agrega un método virtual puro `nombre()` que retorne el tipo de figura.

8.7 Ejercicio 7: Sistema de Vehículos

Diseña una jerarquía de clases donde:

- `Vehiculo` sea una clase base con un método virtual puro `mostrarInfo()`.
- `Auto`, `Bicicleta` y `Camion` sean clases derivadas.
- Cada una muestre distinta información (marca, tipo, carga, etc.).

Crea varios vehículos y guárdalos en un arreglo de punteros a `Vehiculo`. Utiliza polimorfismo para mostrar la información de todos.